

Suveníry

Amaru si kupuje suveníry v zahraničnom obchode. Existuje N **druhov** suvenírov. V obchode je k dispozícii nekonečné množstvo suvenírov každého druhu.

Každý druh suveníru má pevnú cenu. Konkrétne, suvenír typu i ($0 \leq i < N$) má cenu $P[i]$ mincí, kde $P[i]$ je kladné celé číslo.

Amaru vie, že suveníry sú zoradené zostupne podľa ceny, a že ceny suvenírov sú navzájom rôzne. Formálne, platí $P[0] > P[1] > \dots > P[N-1] > 0$. Navyše si zistil presnú hodnotu $P[0]$. O ostatných cenách suvenírov zatiaľ nemá žiadne ďalšie informácie.

Aby si Amaru kúpil nejaké suveníry, vykoná s predajcom niekoľko transakcií.

Každá transakcia pozostáva z nasledujúcich krokov:

1. Amaru podá predajcovi určitý (kladný) počet mincí.
2. Predávajúci zoberie mince do zadnej miestnosti, do ktorej Amaru nevidí. Tam má prázdny stôl. Na ten uloží všetky mince na jednu kôpku.
3. Predávajúci sa postupne pozrie na každý typ suveníru v poradí $0, 1, \dots, N-1$, jeden po druhom. Každý typ suveníru berie do úvahy **práve raz**.
 - Pri zvažovaní suveníru typu i , ak je aktuálny počet mincí v kôpke aspoň $P[i]$, potom
 - predávajúci odstráni z kôpky $P[i]$ mincí a
 - predávajúci položí na stôl jeden suvenír typu i .
4. Od predajcu dostane Amaru všetko, čo skončilo na stole: všetky suveníry, ktoré naň položil, aj všetky mince, ktoré zostali na kôpke.

Upozorňujeme, že pred začiatkom transakcie nie sú na stole žiadne mince ani suveníry.

Vašou úlohou je pomôcť Amaru postupne spraviť nejaké transakcie tak, aby:

- v každej transakcii si kúpil **aspoň jeden** suvenír a
- celkovo kúpil **presne** i suvenírov typu i , pre každé i také, že $0 \leq i < N$. Upozorňujeme, že z toho vyplýva, že Amaru si nikdy nesmie kúpiť žiadny suvenír typu 0.

Amaru nemusí minimalizovať počet transakcií a má neobmedzenú zásobu mincí.

Implementačné detaily

Implementuj procedúru:

```
void buy_souvenirs(int N, long long P0)
```

- N : počet typov suvenírov.
- $P0$: hodnota $P[0]$.
- Táto procedúra bude zavolaná presne raz pre každý testovací vstup.

Vyššie uvedená procedúra môže volať nasledujúcu funkciu vždy, keď chce vykonať transakciu:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : počet mincí, ktoré Amaru odovzdal predajcovi.
- Procedúra vráti dvojicu čísel. Prvým prvkom dvojice je pole L , obsahujúce druhy zakúpených suvenírov (vo vzostupnom poradí). Druhým prvkom je celé číslo R : počet mincí, ktoré Amaru dostal späť po transakcii.
- Číslo M musí spĺňať nerovnosti $P[0] > M \geq P[N - 1]$. Podmienka $P[0] > M$ zabezpečuje, že Amaru nekúpi žiadny suvenír typu 0. Podmienka $M \geq P[N - 1]$ zabezpečuje, že Amaru kúpi aspoň jeden suvenír. Ak tieto podmienky nie sú splnené, tvoje riešenie dostane verdikt `Output isn't correct: Invalid argument`. Všimni si, že na rozdiel od $P[0]$ nie je hodnota $P[N - 1]$ uvedená vo vstupe.
- Procedúra môže byť pre každý testovací vstup volaná maximálne 5000-krát.

Správanie testovača **nie je adaptívne**. To znamená, že postupnosť cien P je pevne stanovená pred zavolaním tvojej funkcie `buy_souvenirs`.

Obmedzenia

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ pre každé i také, že $0 \leq i < N$.
- $P[i] > P[i + 1]$ pre každé i také, že $0 \leq i < N - 1$.

Podúlohy

Podúloha	Body	Ďalšie obmedzenia
1	4	$N = 2$
2	3	$P[i] = N - i$ pre každé i také, že $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ pre každé i také, že $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ pre každé i také, že $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ pre každé i také, že $0 \leq i < N - 1$.
6	33	Žiadne ďalšie obmedzenia.

Príklad

Uvažujme nasledujúce volanie:

```
buy_souvenirs(3, 4)
```

Existujú $N = 3$ druhy suvenírov a $P[0] = 4$. Všimni si, že existujú iba tri možné postupnosti cien P : $[4, 3, 2]$, $[4, 3, 1]$, a $[4, 2, 1]$.

Predpokladajme, že `buy_souvenirs` zavolá `transaction(2)`, a že toto volanie vráti $([2], 1)$. Toto znamená, že Amaru si kúpil jeden suvenír typu 2 a predávajúci mu vrátil 1 mincu. Všimni si teraz, že z týchto informácií si vieme odvodiť, že $P = [4, 3, 1]$, pretože:

- Pre $P = [4, 3, 2]$ by volanie `transaction(2)` vrátilo $([2], 0)$.
- Pre $P = [4, 2, 1]$ by volanie `transaction(2)` vrátilo $([1], 0)$.

Potom môže tvoja funkcia `buy_souvenirs` zavolať `transaction(3)`. Toto volanie vráti $([1], 0)$, čo znamená, že Amaru si kúpil jeden suvenír typu 1 a predávajúci mu vrátil 0 mincí. Doteraz si teda dokopy kúpil jeden suvenír typu 1 a jeden suvenír typu 2.

Nakoniec môže `buy_souvenirs` zavolať napríklad `transaction(1)`. Toto volanie vráti $([2], 0)$, čo znamená, že Amaru si kúpil jeden suvenír typu 2. Dokopy teda má jeden suvenír typu 1 a dva typu 2, čo je presne to, čo požadujeme.

Všimni si, že ako posledné volanie sme tiež mohli použiť aj `transaction(2)`.

Vzorový testovač

Formát vstupu:

```
N
P[0] P[1] ... P[N-1]
```

Formát výstupu:

```
Q[0] Q[1] ... Q[N-1]
```

Hodnota $Q[i]$ je celkový počet zakúpených suvenírov typu i pre každé i také, že $0 \leq i < N$.